COS214 Prac3

JA Rafudeen – u25011503
CL Botha – u25017749
SL Dlamini – u25141300

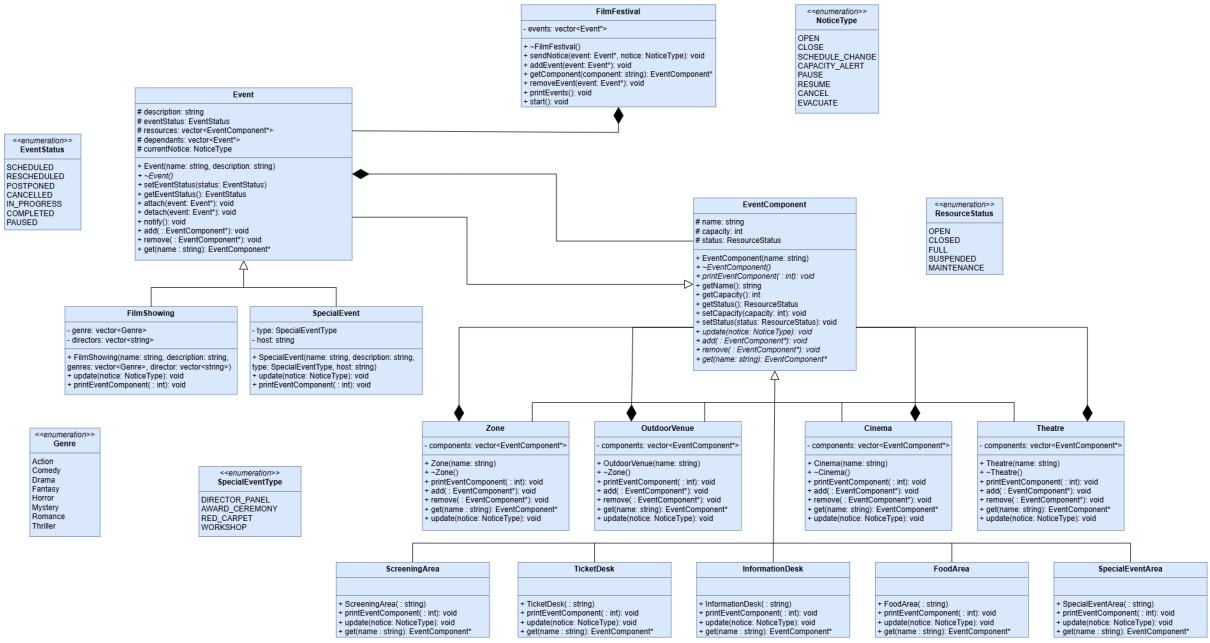
1.1) Our event is a Film Festival, called the South African National Film Festival. It is a place for aspiring filmmakers in South Africa to show off their work, for directors to host panels where they talk about their films, and for nominated films to be given awards. It can take place inside a cinema, a theatre, an outdoor venue, or an area that holds any of those, such as a mall (groupings that are Composite). Within those venues there can be a screening area, a ticket desk, an information desk, a place to buy food, or a place to host the special events such as the awards show (concrete event types).

1.2)
Composite participants:
Component – EventComponent
Composite – FilmShowing, SpecialEvent, Zone, OutdoorVenue, Cinema, Theatre
Leaf – ScreeningArea, TicketDesk, InformationDesk, FoodArea, SpecialEventArea

Observer participants:
Observer – Event
ConcreteObserver – FilmShowing, SpecialEvent
Subject – Event
ConcreteSubject - FilmShowing, SpecialEvent
(Observer is a Subject in our model)

Event is a participant in both, because an Event can hold other Events (composite), and if an Event holding another Event gets cancelled (for example), then it has to notify its child Event to cancel it as well (Observer and Subject).

1.3)



1.4)

a) Our Composites are essentially venues, and venues can hold other venues. A common example is a cinema being inside a mall. As for leaves, a venue can contain a food area, a ticket desk, etc., but those leaves cannot have other leaves or venues. A ticket desk cannot have another ticket desk inside of it.

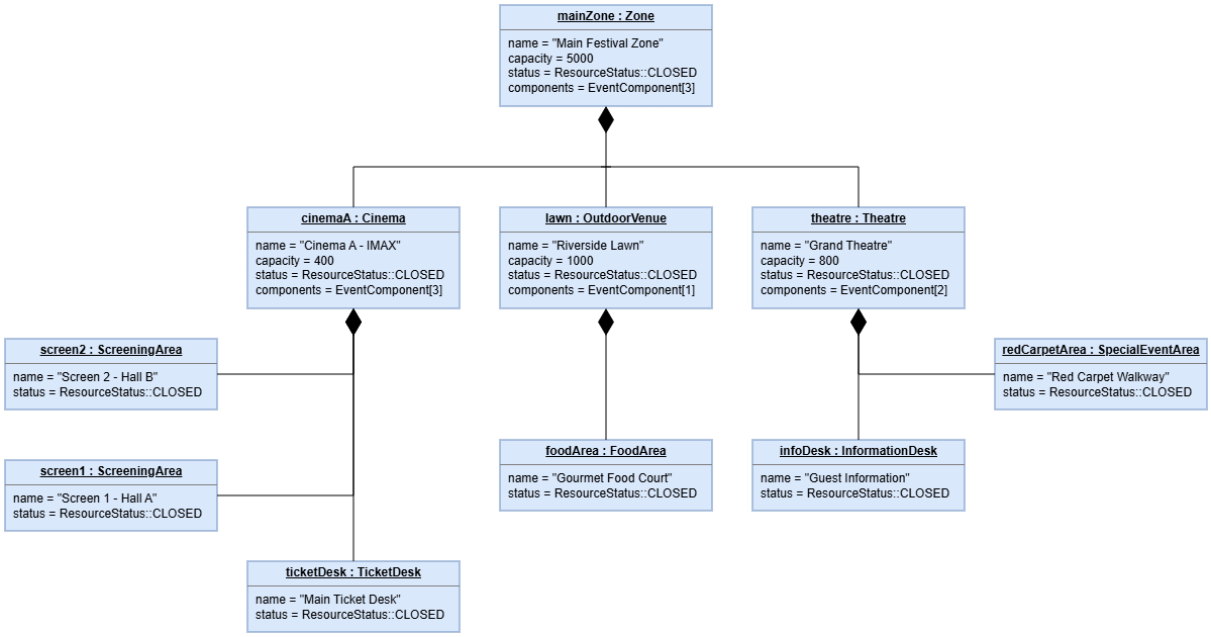
b) Since an Event is also a Composite, it can have other Events inside it. If one parent Event is cancelled/rescheduled, its children would need to be notified of this in order to decide if they should also cancel/reschedule. Hard-coded calls will not work as effectively because the client would need to keep track of which Events need to be notified after one changes, which would become needlessly complicated and easily prone to error.

c) Event components are owned by the EventComponent, which is owned by Event.

d) The client (FilmFestival) owns the Observer reference. The Observer/Subject itself owns a reference to Observer, because in our system the Observer and Subject are the same.

e) An Observer is simply a class whose objects (may) update when a Subject's object does. If an Observer object updates based on another object in the same class, then it just means that the Observer and Subject belong to the same class.

2.3)



3.5) We chose the push approach for our Observer implementation. When the Subject changes its state, the Subject directly passes the relevant information to each registered Observer through update(NoticeType). The pushed information is the NoticeType, which Observers the respond to. We chose push because the Observers will always need to know what NoticeType was passed to the Subject, so it's not necessary to query the Subject to determine what changed. This makes the notification interaction straightforward and allows components to react immediately and differently to the same notice through polymorphism.

The trade-off is that the Subject must decide what information is relevant and include it in the notification. This can make the notification interface more tightly coupled to the information being communicated than the pull approach, where Observers query the Subject themselves.

4.1)

- Assume a FilmShowing Event has a DirectorPanel as a SpecialEvent. If that FilmShowing is cancelled, the associated DirectorPanel should also be cancelled, however, every other FilmShowing should continue as normal.
- Suppose a mall is hosting a FilmFestivals but is showing it at two locations at once, one in a cinema and one outdoors in the parking lot (both inside the mall's property). If a WeatherAlert occurs due to bad weather, the OutdoorVenue should be notified such that it can close and the people inside can be relocated

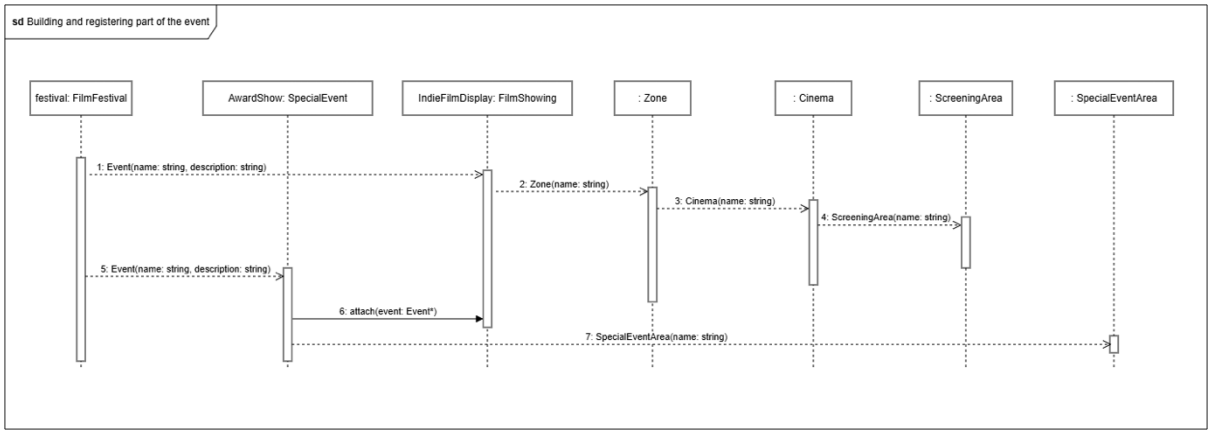
to the Cinema; the Cinema should not close, however it should be aware of the new people coming in and send a CapacityAlert if there is not enough space.

- When a FilmFestival inside of an independent building closes/finishes, then the venues should remain open in case other movies are playing (unrelated to the festival), the TicketDesk, InformationDesk, FoodArea and ScreeningArea remain open, however the SpecialEventArea closes.
- If the FilmFestival is held inside a building used exclusively for Film Festivals, then if there is no active FilmShowing or SpecialEvent happening, the ScreeningArea, SpecialEventArea and TicketDesk should not be open. The InformationDesk and FoodArea may still be open, for if information about a previous/future festival needs to be acquired, or if someone wishes to purchase food.
- If a FilmShowing becomes Paused, then the ScreeningArea should pause and only resume when the FilmShowing resumes. The TicketDesk, InformationDesk and FoodArea remain open and active while this happens. The SpecialEventArea may need a ScheduleChange when this happens, depending on how long the FilmShowing is paused for.

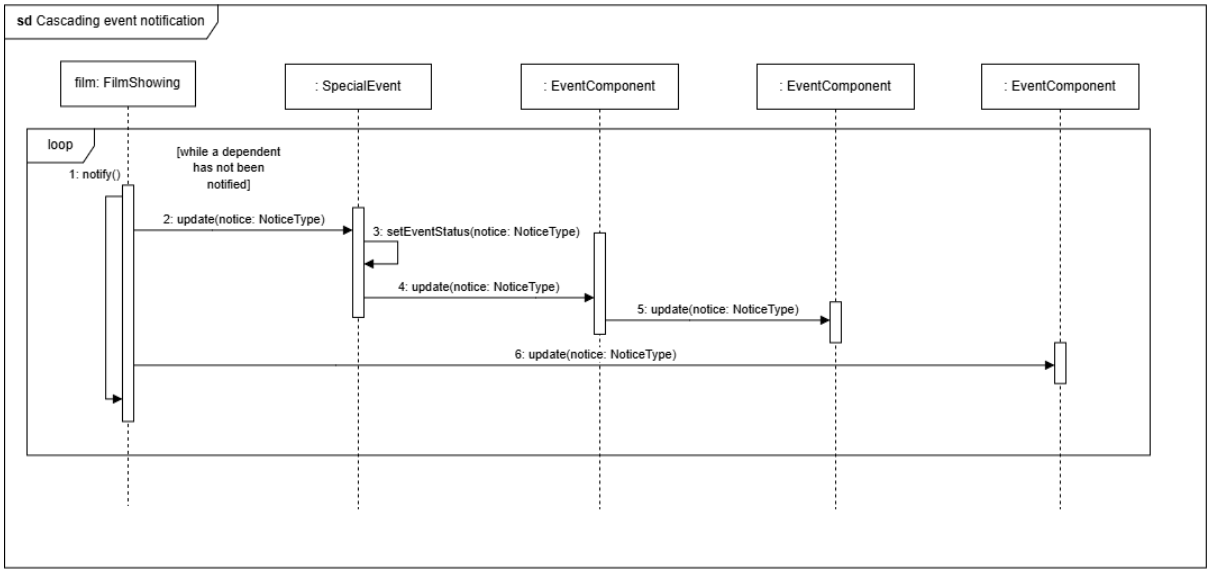
4.4)

- Our Observer/Subject is also a Composite. This was necessary because an Event can contain other Events.
- Our Observer is also our Subject. This was necessary because an update to a parent Event should update the children.
- We added Enumeration classes to various classes, which makes our design more compact, streamlined and efficient. This eliminates human error.

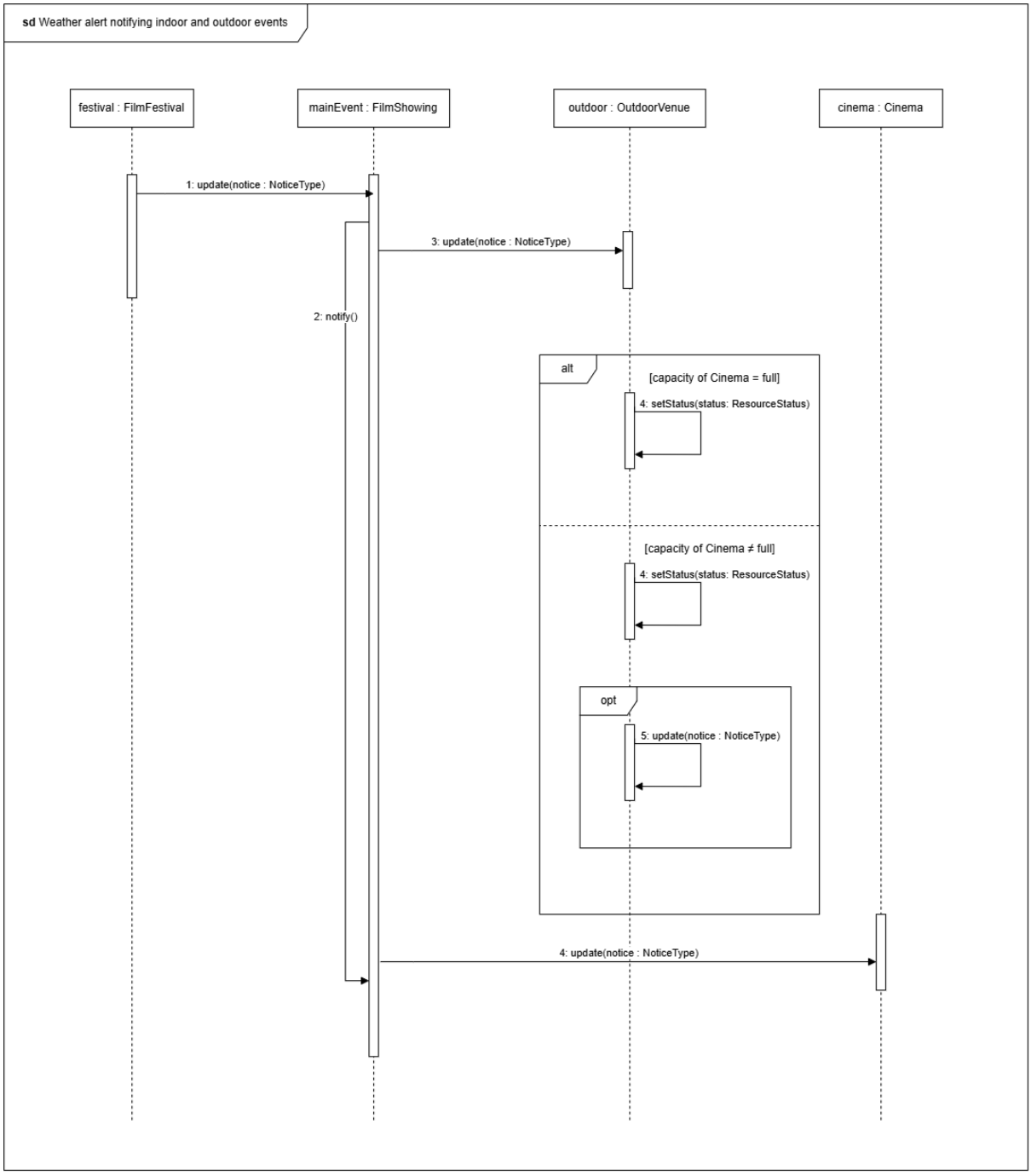
5.1)



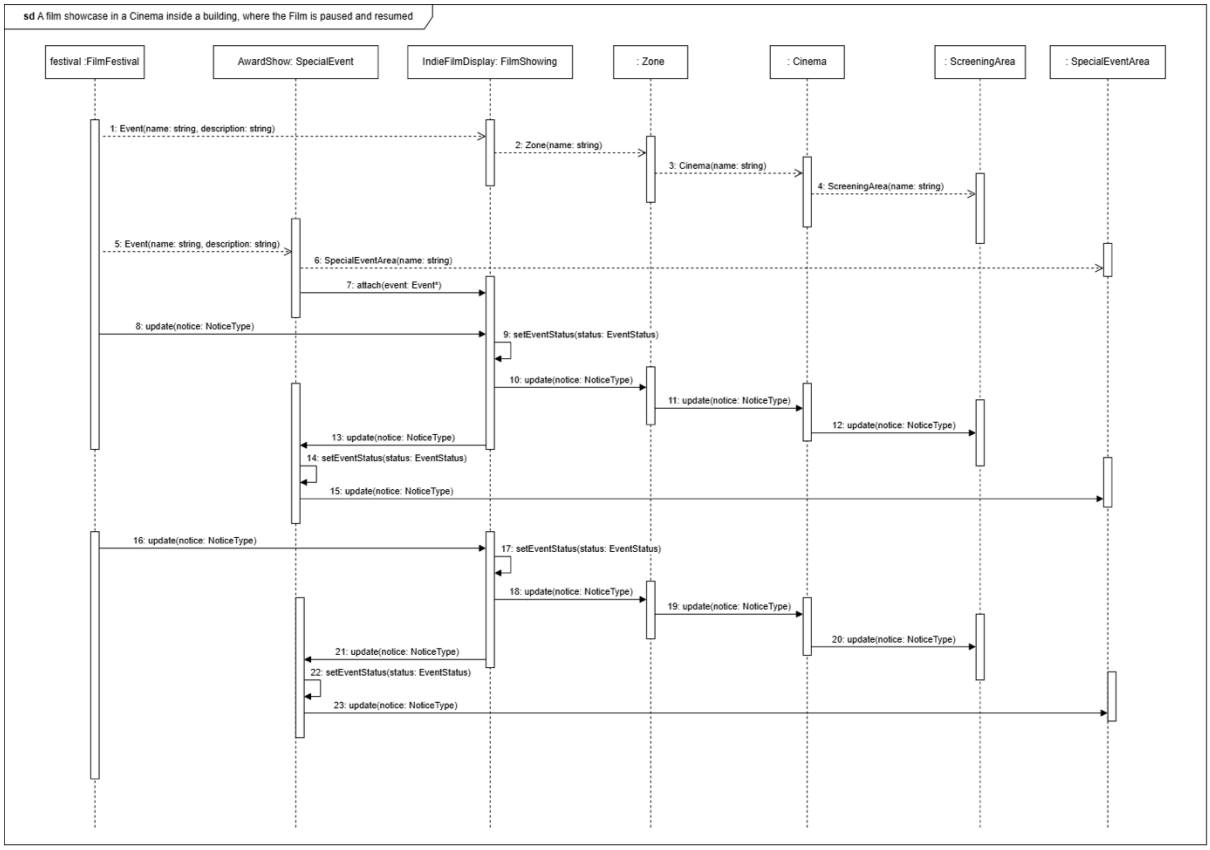
5.2)


sd Cascading event notification
film: FilmShowing
: SpecialEvent
: EventComponent
: EventComponent
: EventComponent
loop
[while a dependent has not been notified]
1: notify()
2: update(notice: NoticeType)
3: setEventStatus(notice: NoticeType)
4: update(notice: NoticeType)
5: update(notice: NoticeType)
6: update(notice: NoticeType)

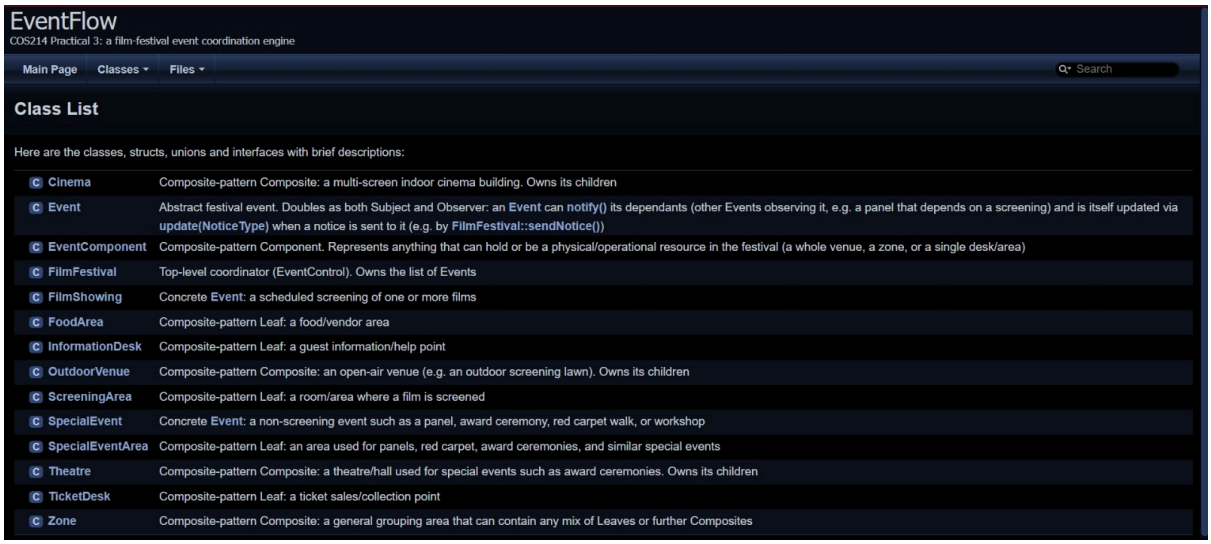
5.3)


sd Weather alert notifying indoor and outdoor events
festival : FilmFestival
mainEvent : FilmShowing
outdoor : OutdoorVenue
cinema : Cinema
1: update(notice : NoticeType)
3: update(notice : NoticeType)
2: notify()
alt
[capacity of Cinema = full]
4: setStatus(status: ResourceStatus)
[capacity of Cinema ≠ full]
4: setStatus(status: ResourceStatus)
opt
5: update(notice : NoticeType)
4: update(notice : NoticeType)

5.4)



Doxygen



Github

https://github.com/KatNikKing/COS-214-Practical_

7.4) The work was essentially divided into coding and designing, with the designing taking two people. Simnikiwe and Junayd were in charge of the system design and diagrams, and Conrad was in charge of the coding/implementation and Doxygen. All members, however, had a say in each aspect of the practical.

Committing helped us collaborate as a member who wants to see update some code can modify the most recent code that was committed. We also collaborated by using the “Issues” tab, which allowed us to remember what needs to be implemented and notify others of bugs/errors that may have been found.
We used a different branch for each team member, and it helped by making it easier to locate certain files that a member was working on. It also helped when changing code, as one member could experiment with the code before pushing it to main, meaning we were not at risk of overriding our already functioning code.